

LC 2258: Escape the Spreading Fire

Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 27, 2026

Amazon | PhonePe | Trilogy

Problem Breakdown

0 → Grass	1 → Fire	2 → Wall (Man and fire can't pass)
-----------	----------	---------------------------------------

- Start from **(0,0)** and reach destination **(m-1, n-1)**.
- We may move to an adjacent grass cell. After our move, every fire cell spreads in all directions.
- **Goal:** Return the **max number of minutes** we can stay in the initial cell before moving and still safely reach the destiny. If impossible, return **-1**.

Core Strategy & Intuition

The fire spreads deterministically. So we can use a **Multisource BFS** to find when exactly a cell catches fire (Pre-calculating fire arrival time for each cell).

Key Idea: Binary Search on Waiting Time

Suppose you decide to wait X minutes and still survive. Then waiting $(X-1)$, $(X-2)$... minutes works too.

Since the answer is monotonic, we can apply **BINARY SEARCH!**

State representation: (row, col, time)

Valid Transition: Only if `arrival time < fire time`.

Important Edge Case: If we are reaching the safehouse $(m-1, n-1)$, we can reach it at the exact same time the fire reaches it (i.e., `arrival time <= fire time`).

C++ Solution Code

Notes: Har ek direction me move krne ki try karo, ek cell me tabhin jao agar waha ab tak fire nahi phuchi. Sbse pehle ek matrix bana lete he which will calc kaun se cell me aag kis time pe lagegi.

[illegible]

```

        q.push({{nr,nc}, currt + 1});
        seen[nr][nc] = 1;
    }
}
}
}
}
return false;
}

int maximumMinutes(vector<vector<int>>& grid) {

    // sbse pehle ek matrix bana lete he which will calc, kaun se cell me aag kis

    n = grid.size();
    m = grid[0].size();

    vector<vector<int>> ftime(n, vector<int>(m, INT_MAX));

    // multisource bfs, multiple cells se aag lagni start ho skti
    // queue will store {{r,c}, time}
    queue<pair<pair<int,int>, int>> q;
    vector<vector<int>> seen(n, vector<int>(m, 0));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 1) {
                q.push({{i,j}, 0});
                seen[i][j] = 1;
            }
        }
    }

    while (!q.empty()) {
        int r = q.front().first.first;
        int c = q.front().first.second;
        int currt = q.front().second;

        q.pop();

        for (auto &it : dir) {
            int nr = r + it.first;
            int nc = c + it.second;

            if (nr >= 0 && nc >= 0 && nr < n && nc < m) {
                if (!seen[nr][nc] && grid[nr][nc] != 2) {
                    seen[nr][nc] = 1;
                    ftime[nr][nc] = currt + 1;
                    q.push({{nr,nc}, currt + 1});
                }
            }
        }
    }
}

```

```
int ans = -1;
int l = 0;
int h = 1e9;

while (l <= h) { // x minute wait pe phuch gaya to x-1, x-2 .... pe bhi ho jaa
    int mid = l + (h - l) / 2;

    if (canreach(grid, ftime, mid)) {
        l = mid + 1;
        ans = mid;
    } else {
        h = mid - 1;
    }
}

return ans;
};
```